

Guide 4

Distributed LLM Inference with llama.cpp: Single- and Multi- Node

2025/2026

1 Introduction

This guide introduces inference serving with llama.cpp ARM nodes. You will run the same model in two modes:

- **Single node:** one llama-server process on one node.
- **Multi node:** one head node plus multiple RPC worker nodes.

The main goals are:

- Understand the deployment structure of llama.cpp server mode.
- Launch inference jobs with Slurm in single-node and multi-node settings.

2 Project Structure

This guide assumes the following tree layout:

```
guide4/  
  container/  
    container-llama.sif  
  single_node/  
    llama_singlenode.sh  
  multi_node/  
    start_rpc_worker.sh  
    run_llama_head.sh  
    submit_cluster.sh
```

Shared model location:

```
/projects/F202500001HPCVLABEPIURE/guides/models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf
```

Directory navigation to reach these files:

```
cd /projects/F202500001HPCVLABEPIURE/guides/guide4
```

Recommended workflow: copy to your personal folder and work there.

```
cd /path/to/your/personal/folder  
cp -r /projects/F202500001HPCVLABEPIURE/guides/guide4 .
```

3 Single Node Inference

The file `single_node/llama_singlenode.sh` launches:

- `llama-server` from `/opt/llama.cpp/build/bin` inside the container.
- Host `0.0.0.0`, port `8080`.
- One model file from the shared absolute path `/projects/F202500001HPCVLABEPICTURE/guides/models/`.

3.1 Submit single-node job

```
cd /path/to/your/personal/folder/guide4/single_node
sbatch llama_singlenode.sh
```

3.2 Slurm settings

```
#SBATCH --partition=normal-arm
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=46
```

The node has 48 cores, but this setup uses 46 cores for `llama.cpp` and leaves 2 cores available for system processes.

4 Multi Node Inference (RPC Workers + Head)

The multi-node flow uses three scripts:

1. `start_rpc_worker.sh`: starts one `rpc-server` per worker node.
2. `submit_cluster.sh`: submits workers, waits until they are running, builds `RPC_NODES`, and submits the head job.
3. `run_llama_head.sh`: script executed by the head submission to start `llama-server` with `--rpc $RPC_NODES`.

4.1 How worker discovery works

Each worker writes metadata to:

```
.rpc-workers/<JOB_ID>.env
```

containing node name, InfiniBand interface, IP, and port.

Worker communication uses IPoIB (IP over InfiniBand), by default through `IB_IFACE=ib0`.

The submit script polls Slurm + metadata files until all workers are running, then creates:

```
RPC_NODES=ip1:50052,ip2:50052,...
```

and submits the head job with this variable exported.

4.2 Submit multi-node job

```
cd /path/to/your/personal/folder/guide4/multi_node
N_WORKERS=4 WORKER_CPUS_PER_TASK=46 ./submit_cluster.sh
```

You can assign fewer cores per worker if needed, for example:

```
N_WORKERS=4 WORKER_CPUS_PER_TASK=32 ./submit_cluster.sh
N_WORKERS=4 WORKER_CPUS_PER_TASK=24 ./submit_cluster.sh
```

4.3 Slurm settings

Both head and workers are configured for:

```
#SBATCH --partition=normal-arm
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=46
```

As in single-node mode, the default 46 CPU allocation leaves 2 cores free on a 48-core node for background processes.

4.4 Latency and Throughput Tuning

In `multi_node/run_llama_head.sh`, you can test additional llama-server options to improve latency/throughput trade-offs:

```
CTX_CHECKPOINTS="${CTX_CHECKPOINTS:-32}" # Default is 32
BATCH_SIZE="${BATCH_SIZE:-2048}" # Default is 2048
UBATCH_SIZE="${UBATCH_SIZE:-512}" # Default is 512
NUMA_MODE="${NUMA_MODE:-numactl}" # Default is deactivated. Options: "distribute", "isolate", "numactl"
```

Then, in the final `exec singularity exec ... llama-server` command, enable the matching flags:

```
--ctx-checkpoints "$CTX_CHECKPOINTS" \
--batch-size "$BATCH_SIZE" \
--ubatch-size "$UBATCH_SIZE" \
--numa "$NUMA_MODE"
```

In the current script, these lines are commented; uncomment both the variable definitions and the command-line options when testing.

Recommended practice: Change one parameter at a time and compare both latency and throughput across runs.

5 Testing the Inference Server

After the head server starts, check the logs (`head_<id>.out`) to identify the node and verify that port 8080 is active.

Option 1 (direct access from Deucalion nodes): test the API directly on the head node.

```
curl -X POST http://<HEAD_NODE>:8080/completion \
-H "Content-Type: application/json" \
-d '{"prompt": "Explain what distributed inference is.", "n_predict": 128}'
```

Option 2 (local browser via SSH tunnel): access the llama.cpp graphical interface from your PC.

```
ssh <your_username>@login.deucalion.macc.fccn.pt -L 8080:<head_node>:8080
```

Then open in your browser:

```
http://localhost:8080
```

6 Benchmark Table (Single Node vs Multi Node)

Use a table like the one below to record your benchmark runs and compare the latency/throughput across multi-node configurations.

Acronyms used in the table headers:

- CPU/T: CPUs per task.

- CTX_CKPT: context checkpoints (--ctx-checkpoints).
- BATCH: batch size (--batch-size).
- UBATCH: micro-batch size (--ubatch-size).
- NUMA: NUMA mode (--numa).
- tok/s: tokens per second.
- Avg req (s): average request time in seconds.

Table 1: Distributed LLM Benchmark for Multi-Node Inference

Workers	CPU/T	CTX_CKPT	Batch	Ubatch	NUMA	Prompt tok/s	Eval tok/s	Avg req (s)
–	–	–	–	–	–	–	–	–

Replace values with your measured results from head_<id>.out logs and API test timings.

7 Questions

- Based on your benchmark table, what trade-off do you observe between Prompt tok/s, Eval tok/s, and Avg req (s) across different configurations?
- How do N_WORKERS and WORKER_CPUS_PER_TASK affect scaling efficiency and resource usage?
- Which tuning parameter had the largest impact in your tests (CTX_CHECKPOINTS, BATCH_SIZE, UBATCH_SIZE, or NUMA_MODE) and why?

8 Learning Outcomes

This guide demonstrates how to deploy llama.cpp inference in both single-node and multi-node Slurm jobs.